

L AFC-Evict: A Large-Scale Counterfactual Benchmark for Learned Cache Eviction

Soroush Vahidi^{a,*}

^a*New Jersey Institute of Technology, Newark, NJ, USA*

Abstract

A single trace replay evicts one candidate per decision and never reveals the counterfactual information needed to fully evaluate alternative eviction choices. This paper introduces L AFC-Evict, a reusable artifact that constructs this counterfactual evidence at scale: for every resident candidate at every full-cache miss, it provides a finite-horizon counterfactual miss-cost label. The dataset is computed from real cache traces across five workload families and four cache capacities, packaging aligned candidate, decision, and pairwise task views. We systematically characterize this supervision surface and its predictive value. First, the target is frequently highly degenerate: over two-thirds of all decisions are fully tied in aggregate, though discriminativeness varies predictably by capacity and look-ahead horizon. Even so, in regimes where the offline signal separates candidates, offline policy rankings correspond positively with true closed-loop miss-ratio performance. We show this offline-to-online linkage is robust to leave-one-family-out evaluation, family-level block permutations, and variations in the specific continuation policy assumed after a forced eviction. However, offline supervision is not a panacea: a separate frozen learned selector, trained on this data and evaluated in true closed-loop replay, generally underperforms simple robust heuristics like LRU and SIEVE (winning in only 1 of 10 evaluated settings). This negative check confirms that offline predictive structure does not automatically guarantee closed-loop cache-miss gains, underscoring that static supervision complements rather than replaces closed-loop validation. L AFC-Evict releases this candidate-level supervision to standardize and evaluate

*Corresponding author.

Email address: `sv96@njit.edu` (Soroush Vahidi)

future learned-eviction methods.

Keywords: learned cache eviction, benchmark datasets, counterfactual supervision, cache replacement

1. Introduction

When a cache request misses and the cache is full, a system must pick one resident object to evict before admitting the requested object. Evaluating whether a given eviction decision was a good one requires knowing what would have happened to each candidate victim if it, rather than another object, had been evicted. That information is not directly observable from a single trace replay, because only one candidate is actually evicted at each decision. This paper studies a reusable way to construct that counterfactual evidence at scale. It evaluates when the evidence is informative, when it degenerates, how it relates to actual closed-loop cache performance, and how sensitive it is to one of its own modeling assumptions.

L AFC-Evict materializes, for real cache traces, a per-decision, per-candidate finite-horizon counterfactual miss cost. For each object that could have been evicted at a full-cache miss, the cost counts how many additional misses would occur over the next H requests if that object, specifically, had been the one evicted. This candidate-level quantity is stored as `y_loss` (a convenience alias `y_value = -y_loss` is also provided; see Section 8). Section 2 works through one full decision by hand – resident objects, a forced eviction, a future request window, and the resulting candidate costs, including a case where two candidates tie for optimal – before any aggregate statistic is introduced. This grounds every quantity used later in the paper in a concrete example first.

The contribution of this paper is not a new eviction policy. It is an evaluation of a reusable offline supervision surface against closed-loop cache-policy behavior, organized around five research questions: **(RQ1) Is the target discriminative between candidates?** We find massive tie-degeneracy overall, but discriminativeness varies predictably by workload, capacity, and horizon. **(RQ2 & RQ3) Does the offline ranking correspond to actual closed-loop performance, and what drives it?** Offline and closed-loop orderings agree in most regimes where the offline signal is not near zero, driven heavily by capacity-scale locality in the underlying trace. **(RQ4) How sensitive are the labels to the assumed continuation policy?**

A pre-registered study finds the canonical labels robust to MRU and random continuation, though random continuation is measurably more perturbing. **(RQ5) What does this mean in practice?** Offline supervision complements rather than replaces closed-loop evaluation; we provide concrete, evidenced guidance for when the offline signal can be trusted. Taken together, these answers quantify when finite-horizon counterfactual supervision is aligned – or measurably misaligned – with closed-loop policy behavior.

Every quantitative claim in this paper traces to a specific, independently validated evidence artifact. Sections 8 through 15 and the limitations discussion in Section 17 state the exact scope, sample sizes, and caveats attached to each result. The underlying release also exposes candidate-level, decision-level, and pairwise derived views of the same supervision at production scale (Table A.12), intended for reuse by other researchers studying learned cache eviction.

Section 18 positions this contribution precisely against learned-caching, benchmark, and offline-vs-deployed-evaluation literature. In brief, existing public cache-replacement resources do not provide a reusable candidate-level counterfactual supervision surface at this scale together with a direct evaluation of its relationship to closed-loop cache-policy behavior.

The rest of the paper proceeds as follows. Section 2 gives the worked example. Sections 3–7 describe construction, schema, and release validation. Section 8 defines the benchmark tasks. Section 9 reports target discriminativeness and, together with the closed-loop production evaluation, addresses offline/closed-loop correspondence and continuation-policy robustness. Section 15 reports a separate frozen learned-policy closed-loop check. Section 17 states limitations, followed by related work and conclusion.

2. Background and a Worked Example

At each full-cache miss, a caching system must evict one resident object before admitting the requested one. Classical policies such as LRU, ARC [1], and LIRS [2] operationalize this choice through recency/frequency structure; more recent heuristics such as SIEVE [3] and S3-FIFO [4] revisit that structure for scan-resistance and simplicity. A recent hybrid line tunes such heuristics’ own parameters with a learned control plane rather than replacing them outright: LAH, instantiated as S4-FIFO over an S3-FIFO data plane, reports robustness and interpretability gains from this decoupling [5].

Learned or learning-augmented approaches – for example LeCaR, LRB, Parrot, and GL-Cache [6, 7, 8, 9] – instead attempt to predict which eviction will lead to lower future miss cost. CACHEUS, 3L-Cache, and HALP take the same general approach [10, 11, 12], typically training on labels each system derives for itself from raw traces. This makes cross-paper comparison hard: different papers use different traces, different label constructions, and different evaluation protocols, so a reported improvement is often not attributable to the policy alone.

LAFC-Evict addresses the label-construction half of that problem by materializing a reusable, candidate-level counterfactual label directly from real traces, independent of any specific learned method. The rest of this section works through exactly what that label is, using a small, fully hand-checkable example (full derivation in the paper’s reproducibility artifact); Section 3 then defines the same quantities formally for the production-scale dataset.

2.1. A worked eviction decision

Suppose a cache holds three objects at capacity 3, ordered from least-recently-used to most-recently-used: A, B, C. A request arrives for object D, which is not resident: a *cache miss*. The cache is full, so admitting D requires evicting exactly one resident object. A, B, and C are the three *candidate victims* for this *eviction decision*.

To score each candidate, we look ahead a finite *horizon* of $H = 3$ future requests, which turn out to be B, E, F. For each candidate, we ask: if this candidate, and no other, had been evicted, and the cache had then continued to evict under a *continuation policy* (LRU, the canonical choice throughout this dataset) for the next H requests, how many of those requests would miss? This defines the candidate’s *counterfactual miss cost*, stored as `y_loss` (a convenience alias `y_value = -y_loss` is also provided; see Section 3).

Figure 1 works through all three resulting counterfactual worlds by hand. Evicting A costs 2 future misses (B hits because A’s eviction did not affect B; E and F both miss because they are new objects). Evicting B costs 3 (B itself is now a miss, plus E and F). Evicting C also costs 2, by the same reasoning as A. **The optimal candidate set for this decision is $\{A, C\}$, a tie** – both evictions are equally good by this measure, and only evicting B is strictly worse. This is not a contrived edge case: across the full production dataset, no audited decision has a strict unique winner (Section 9), so tie handling of exactly this kind is the common case, not the exception.

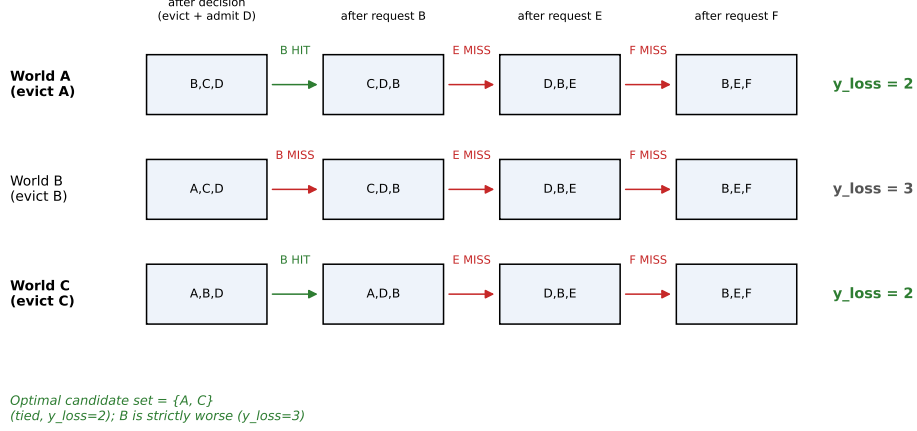


Figure 1: The worked example: three counterfactual worlds (one per candidate victim), each replayed forward under LRU continuation for the next $H = 3$ requests. A and C tie for optimal ($y_loss = 2$); evicting B is strictly worse ($y_loss = 3$).

2.2. From one decision to the released views

Grouping every candidate row by its decision id gives the *decision view* (candidate count, optimal-set membership, tie flags, regret summaries). Deriving pairwise preference labels directly from the candidate-level y_loss values – e.g. this decision yields (A,B): A preferred, (A,C): tie, (B,C): C preferred – gives the *pairwise derived view*. The pairwise view introduces no new ground truth: every label in it is a function of the candidate view already described. This is a narrower claim than systems such as HALP, which train directly on preference-comparison feedback as their primary learning signal [12]: LAFC-Evict’s pairwise view is instead a deterministic reformatting of the already-defined candidate-level y_loss values, released for compatibility with pairwise/ranking methods rather than as an independent source of supervision (Section 8 discusses why a derived pairwise view is still useful for certain modeling and diagnostic purposes). A single full-cache miss with k admissible candidates therefore appears in three aligned forms: k candidate rows, one decision-group summary, and up to $k(k - 1)/2$ within-decision pairwise comparisons.

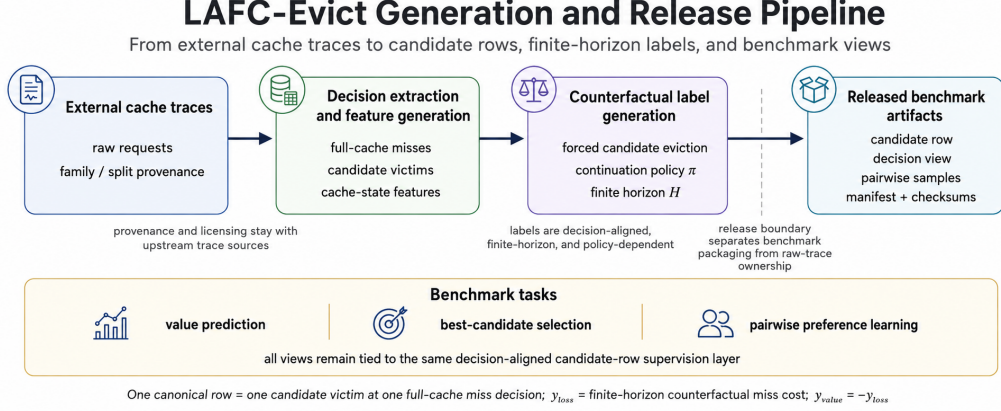


Figure 2: LAFC-Evict generation and release pipeline. External cache traces are converted into decision-level candidate rows, finite-horizon counterfactual labels, and released benchmark views for value prediction, best-candidate selection, and pairwise preference learning.

3. Evaluation Methodology: LAFC-Evict Construction

This section defines the scientific abstraction underlying LAFC-Evict before describing how it is packaged for release (the end-to-end pipeline is summarized in Figure 2). A *trace* is a sequence of *requests* (accesses) to objects. A *cache state* is the set of objects resident in a bounded-capacity cache at a point in the trace. When a request misses and the cache is full, the system faces an *eviction decision*: choose one resident object, a *candidate victim*, to evict before admitting the requested object. LAFC-Evict treats every such decision, across every trace, capacity, and horizon evaluated, as one supervised unit; Section 5 defines the exact formal quantities (finite-horizon counterfactual loss, optimal candidate set, regret) built on top of this abstraction, following directly from the worked example in Section 2.

The cache abstraction used throughout this paper is intentionally narrow and stated explicitly here rather than left implicit: objects are unit-sized (no byte-weighted or variable-size accounting), capacity is counted in objects, and admission of the requested object is unconditional (eviction is never avoided by refusing admission). Section 17 discusses the consequences of this scope boundary; every claim in this paper should be read as bounded by it.

The evaluated dataset covers five trace families – `cloudphysics`¹, `metacdn`, `metakv`, `twemcache`, and `wiki2018` – chosen to span a CDN workload, a key-value workload, two additional production-derived workloads, and one intentionally included negative/control workload (`wiki2018`; Sections 9 and 12 show why it degenerates and treat that as a useful, explained result rather than a benchmark defect). Two further families, `brightkite` and `citibike`, are excluded from the open release subset for redistribution reasons; this is a release-packaging boundary, not a claim that the underlying generated corpus is limited to five families. This five-family evaluated corpus is now publicly available as the LAFC-Evict v1.0 release on the Hugging Face v1.0 revision, while the older v0.3 `wiki2018`-only revision remains available separately; the Data Availability statement details the redistribution clearance and hosting status of the evaluated families. Not every family appears in every split (e.g. `metacdn` contributes train/validation but no test decisions); split assignment is produced upstream of this release package and is normalized and validated, not assigned, here.

The evaluation in this paper is organized to satisfy four requirements:

1. **Decision alignment.** Preserve within-decision candidate competition rather than treating candidates as i.i.d. points.
2. **Task multiplicity.** Support pointwise (candidate-level), decision-level, and pairwise views of the same underlying supervision.
3. **Closed-loop grounding.** Evaluate the offline labels against actual closed-loop cache-policy behavior rather than presenting them as self-validating (Sections 10 and 11).
4. **Reproducibility.** Expose validated, checksummed evidence for every reported number (Section 7).

The canonical workload and evaluation parameters are summarized in Table 1. When reading and interpreting the results, the following trace-specific design nuances should be kept in mind:

¹`cloudphysics` is a historical internal family identifier retained in project manifests, evidence records, and code for reproducibility. The underlying workload is Alibaba Cloud’s Elastic Block Storage production block-storage trace, not the VMware/CloudPhysics dataset. Reader-facing prose below refers to this workload as “alibaba-block.”

Table 1: Canonical evaluation protocol and workload specifications per trace family. Every family has release capacities of $\{32, 64, 128, 256\}$ and horizons of $\{4, 8, 16\}$, while closed-loop evaluation focuses on capacities $\{32, 128\}$.

Family	Workload	Splits	CL Split	Scored Warm-up / Init Win- dow	
alibaba-block	Block Storage	train/val/test	test	2 chunks (8,192 req.)	Warm
metacd	CDN Cache	train/val	validation	3 chunks (20,480 req.)	Warm
metakv	Key-Value	train/val/test	test	1 chunk (4,096 req.)	Cold Start
twemcache	Memcached	train/val/test	test	2 chunks (8,192 req.)	Warm
wiki2018	Wikipedia	train/val/test	test	1 chunk (4,096 req.)	Warm

- **MetaCDN Validation Status:** Tier-1 closed-loop evidence for MetaCDN uses validation-window chunks because no Tier-1 test-split chunks are available at these capacities. The separate learned-policy campaign evaluates MetaCDN under its own independent held-out protocol.
- **MetaKV Cold Start:** MetaKV is evaluated starting from an empty cache (a cold start) with no leading unscored warm-up, whereas the other four workload families continuous-warmup from preceding unscored trace prefixes.
- **Wiki2018 Negative Control:** Wiki2018 is a deterministic zero-reuse negative-control workload where all candidate eviction options result in identical miss costs (100% tie-degenerate, zero random regret), acting as a baseline verification of cache boundaries.

Detailed release characteristics, including candidate scale (277M rows) and exact split breakdowns, are provided in Appendix A (Tables A.12 and A.13).

4. Experimental Methodology

This section consolidates the methodological choices used across every result reported in Sections 9–13 so that scope and caveats are stated once rather than scattered.

Workloads, capacities, horizons. All results use the five families defined in Section 3 (alibaba-block, metacdn, metakv, twemcache, wiki2018). Offline target-discriminateness results (Section 9) span all four evaluated capacities (32, 64, 128, 256) and all three horizons ($H \in \{4, 8, 16\}$) in the reusable release. A separate validated long-horizon sensitivity experiment reuses canonical H16 and extends the same five-family, four-capacity characterization to $H \in \{32, 64, 128\}$; it is an analysis extension, not a change to the public release scope. Closed-loop, linkage, and mechanistic results (Sections 10–12) use capacities 32 and 128. The sampled continuation study in Section 13 also uses capacities 32 and 128, while the full-population MRU continuation census additionally covers capacities 64 and 256. This is because the Tier-1 production closed-loop evaluation, on which the linkage and mechanistic findings depend, is scoped to capacities 32 and 128 (the primary release also preserves the full four-capacity scope for offline use), which is stated here once rather than repeated in each section.

Split terminology and two family-specific caveats. Split assignments are inherited from the frozen upstream generation metadata rather than reassigned by the release pipeline; test windows are chronologically later where available and represent a temporally held-out scoring window, whereas “validation” denotes a window used during earlier development that is not temporally held out in the same sense. The exact scored chunks, request counts, and protocol settings are summarized centrally in Table 1. Two caveats apply and recur throughout the paper: (i) **metacdn’s closed-loop evidence is validation-window evidence, not test-window**, for both of its evaluated capacities; every result that includes metacdn should be read with this in mind, and Section 11 reports what changes if metacdn is excluded. (ii) **metakv is the only family whose scored window begins at an empty cache** (a cold start, identifiable from the identity $evictions = misses - capacity$ holding exactly for metakv and no other family); Section 12 discusses this as a plausible, not established, contributor to metakv’s one policy-ranking exception.

Policies and seeds. Closed-loop replay is evaluated for LRU, MRU, SIEVE (all deterministic, one run each), and random (20 CRN seeds, seeds 0–19; mean and per-seed standard deviation reported). SIEVE has no offline counterpart in the labels described in Section 5 and is therefore closed-loop-only evidence throughout – it is not part of any offline-vs-closed-loop correspondence claim, but is still reported as a closed-loop reference point (Section 10). This Tier-1 comparator set is diagnostic rather than a policy competition: LRU and MRU probe recency sensitivity, random probes the tie-adjusted floor, and SIEVE adds one lightweight modern reference point, against the offline target – it is not a claim of comparison against the full classical and learning-augmented policy landscape surveyed in Section 18.

Offline metrics. Per decision: all-tied fraction, unique-winner fraction, mean/median optimal-set fraction, decision-weighted random-optimal probability, and random regret (mean and tail percentiles), all computed per (family, capacity, horizon) stratum before any pooling (Section 9).

Closed-loop metrics. Miss ratio per (family, capacity, policy), and, for random, its across-seed mean and standard deviation; relative miss-ratio difference vs. LRU is reported as the primary comparison statistic (Section 10).

Offline↔closed-loop linkage metrics. Sign concordance between the offline regret gap and the closed-loop miss-ratio gap (MRU-vs-LRU and random-vs-LRU only, since these are the two pairs with both an offline and a closed-loop side), plus Pearson/Spearman/Kendall correlation between the

two gaps’ magnitudes, computed per horizon on the $n = 10$ (family, capacity) cells (Section 11). $n = 10$ is small and is treated as such throughout: correlations are reported with their exact n , never as population-scale claims.

Continuation-sensitivity metrics. Optimal-set Jaccard overlap, cross-continuation regret (in misses), and strict-reversal fraction, comparing the canonical LRU-continuation label against MRU- and mean-random-continuation (10 CRN seeds) relabelings of the same decisions, on a pre-registered, stratified 5,000-primary + 500-diagnostic decision sample at capacities 32 and 128, plus a full-population MRU census over all five families, capacities 32/64/128/256, and horizons 4/8/16 (Section 13). The primary statistical unit here is the *decision* (not the candidate row or the continuation seed): every reported sampled-study statistic is a decision-level cluster statistic, and the two reported bootstrap confidence intervals (one per continuation policy; Section 13) resample at the decision level; the MRU census is reported as a complete population enumeration, not an inferential sample.

Tie-aware analysis. Every metric above that could be silently distorted by ties (optimal-set membership, pairwise preference, concordance) is computed with ties as a first-class outcome rather than broken arbitrarily; Section 2’s worked example shows the base case this generalizes from.

Provenance and validation strategy. Every number in this paper traces to a validated evidence record. The closed-loop, linkage, mechanistic, and continuation analyses were checked by reproducible scripts and, where applicable, by independent recomputation of headline values (Section 7). No number in this paper is read from a partial or unvalidated intermediate run; the full-population MRU continuation census is used only after complete-record validation and an independent recheck.

5. Counterfactual Label Generation

The benchmark label is defined on candidate victims rather than on completed policies. Let $r_1, r_2, \dots$ denote a request sequence, and let t index a request r_t that misses when the cache is full. Write S_t^- for the cache state immediately before the eviction choice; the candidate set $C_t = S_t^-$ is every resident object, so $|C_t|$ equals the cache capacity (32, 64, 128, or 256 objects in the evaluated release), counted in objects rather than bytes, with upstream object sizes not separately modeled. For any candidate $c \in C_t$, consider the intervention that evicts c , admits the missed request into the freed slot, and then follows a fixed continuation policy π for the next H requests, resolving

every subsequent miss according to π ; in the evaluated release, π is LRU continuation rather than a Belady/offline-optimal rule. The resulting finite-horizon counterfactual loss counts those subsequent misses, so every miss contributes unit cost:

$$L_{t,H}^\pi(c) = \#\{\tau \in \{t+1, \dots, t+H\} : r_\tau \text{ misses in the induced trajectory}\}. \quad (1)$$

The release stores this quantity as `y_loss`, and it stores the corresponding value label as `y_value` = $-\text{y_loss}$. The triggering miss at time t is common to all candidates and is therefore not the decision-discriminating part of the label.

Both columns are retained in the schema, but they are not independent information: `y_value` is a deterministic sign-flip of `y_loss`, computed by the same generation step. The two exist because downstream modeling code is written against either a minimization convention (`y_loss` as a cost/regret target, the convention used throughout the rest of this paper and the more directly interpretable of the two – a raw count of misses over the horizon) or a maximization convention (`y_value` as a value/utility target, convenient for argmax-based candidate selection or value-function-style formulations). This paper treats `y_loss` as canonical and uses `y_value` only where a specific baseline is more naturally stated as a maximization, never reporting the same statistic under both names.

This label has two consequences for the paper. First, it creates a common numerical target for regression-style benchmark tasks. Second, it induces structured decision-level objects. For a fixed decision (t, S_t^-, C_t) , the benchmark-optimal candidate set is

$$C_t^\star = \arg \min_{c \in C_t} L_{t,H}^\pi(c), \quad (2)$$

and the per-candidate regret is

$$\Delta_{t,H}^\pi(c) = L_{t,H}^\pi(c) - \min_{c' \in C_t} L_{t,H}^\pi(c'). \quad (3)$$

These quantities define best-candidate selection targets, pairwise preferences, and tie structure within a decision.

This supervision is a generated counterfactual label family, not an upstream annotation. It is finite-horizon, not asymptotic. It is tied to a continuation policy, not a universally optimal control objective. The label is

therefore decision-aligned for the benchmarked eviction choice, but it is not an offline-optimal label in the sense of solving the full future control problem. These boundaries are central to the scientific positioning of the paper.

6. Dataset Schema and Views

The canonical candidate schema currently consists of decision metadata, candidate identity, split information, labels, and feature columns. Each row corresponds to exactly one candidate victim at exactly one full-cache miss *eviction event*, scored under one horizon. Trace identity and split metadata are recorded through `trace_name`, `trace_family`, `dataset_source`, and `split`. Cache configuration is recorded through `capacity` and `horizon`. `decision_id` identifies the underlying eviction event (trace, capacity, and request position) and is shared by every horizon at which that same event is scored; `decision_t` gives its position in the request stream. Candidate identity is represented through the field `candidate_page_id`. The labels are `y_loss`, the finite-horizon counterfactual miss cost after forcing that candidate eviction under the fixed continuation policy, and `y_value = -y_loss`. Feature columns cover request context, candidate context, predictor/LRU signals, cache summary statistics, and recent request or hit rates.

Terminology used consistently for the rest of this paper: a *candidate row* is one row of the base schema above (one candidate at one eviction event, one horizon). A *decision* (interchangeably, *decision-horizon instance*) is the group of candidate rows sharing the same `decision_id` and `horizon` – since the same eviction event is independently scored at every evaluated horizon, one eviction event contributes one decision per horizon, not one decision overall. Reported decision counts (e.g. Table A.12) are therefore decision-horizon counts, pooled across all evaluated capacities and horizons, not counts of distinct physical eviction events.

Beyond candidate rows, the release defines two derived views. The *decision view* aggregates all candidate rows that share a decision identifier and horizon and summarizes each resulting decision with candidate count, minimum loss, maximum value, the set of optimal candidate IDs, tie count, and regret summary statistics. The *pairwise view* compares candidate pairs from the same decision (same `decision_id` and horizon) and derives pairwise preference labels from the ordering of their `y_loss` values, with ties when the losses are equal. The evaluated open release does not ship the full pairwise view; it ships a capped pairwise sample of 1,000,000 rows instead.

Table 2: Compact summary of candidate-row schema groups and derived-view fields.

Column group	Notes
Decision metadata	family, capacity, horizon, split, IDs
Candidate identity	candidate page ID
Labels	<code>y_loss</code> , <code>y_value</code>
Feature groups	request, candidate, cache, disagreement, rate
Derived decision fields	tie and regret summaries
Derived pairwise fields	A/B labels and regret differences

Operationally, the schema can be read as a layered projection over the same eviction events: candidate rows are the atomic supervised objects, the decision view groups those rows by `decision_id` and horizon, and the shipped pairwise sample keeps only selected within-decision comparisons. Table 2 summarizes those column groups and derived fields directly in manuscript form.

7. Release Construction and Validation

The release pipeline is designed to package existing generated candidate rows into a benchmark artifact without requiring raw traces in the release package. The builder uses an out-of-core DuckDB path to ingest candidate shards, normalize split values, emit partitioned Parquet candidate rows, derive the decision view, optionally create a capped pairwise sample, and write release metadata and checksums.

The validation surface is part of the benchmark contribution: reproducible scripts verify candidate-partition schema completeness, split-set consistency, family inclusion/exclusion, within-decision duplicate and metadata consistency, decision-view schema/count consistency, and checksum integrity.

The evaluated release package passed this full validation suite via those reproducible scripts, independent of unpublished hosting details and superseded internal status notes; this validates the release package’s construction, not final public hosting or DOI completeness.

Table 3 summarizes the release-package components used in the evaluation: candidate rows, derived benchmark views, manifest metadata, and checksums.

Table 3: Release-package components for the evaluated benchmark snapshot. Locations are relative to the release root.

component	release location
candidate rows	data/candidate_rows/
decision view	data/decision_view/decision_view.parquet
pairwise sample	data/pairwise_sample/pairwise_sample.parquet
release manifest	metadata/release_manifest.json
checksums	metadata/checksums.sha256

7.1. Provenance of the evaluation results in this paper

Beyond the released dataset above, every result reported in Sections 9–13 traces to an independently validated analysis record. These records cover target discriminativeness, Tier-1 closed-loop behavior, offline↔closed-loop linkage, mechanistic workload analysis, sampled continuation sensitivity, and the full-population MRU continuation census. Each record includes reproducible scripts and, where applicable, an independent recomputation of headline values. The manuscript reports only results whose validation and recheck steps completed.

Two label sources must not be conflated: released labels use canonical LRU continuation (Section 5); the MRU/random *relabelings* used for the sensitivity study (Section 13) are diagnostic only, not an alternative release.

The population-scale MRU continuation census is complete evidence. The census covers all 2,363,286 decision-horizon records in the evaluated corpus and passed an independent recheck before its summaries were imported into this manuscript. The raw 1.6 GB census output is preserved separately; the manuscript uses only compact summaries, validation reports, manifests, and Figure 7 source data.

A known, unresolved provenance limitation: the exact historical commit provenance of the full-release label generator is incomplete. This does not affect the validity of the validated analyses above, each of which independently records its own generation context, but it means the full historical lineage of the generator code itself is not completely reconstructable from repository history alone.

8. Benchmark Tasks and Baselines

LAFC-Evict supports four benchmark tasks from one release.

Candidate-level value prediction.. Predict `y_loss` or `y_value` from a single candidate row – the pointwise view of finite-horizon counterfactual eviction quality (Table 7).

Best-candidate selection.. Given all candidate rows for one decision, identify a member of the minimum-loss set. This task can be framed as ranking or decision-level classification and connects most directly to the actual cache-eviction choice because it evaluates whether a method selects a benchmark-optimal victim within that decision. We call a decision **discriminative** when `optimal_candidate_count < candidate_count`, i.e. at least one candidate is strictly worse than the best candidate in that decision; on any other (all-tied) decision every candidate is optimal by construction, so no selector’s choice can affect realized regret there. Because of this, all-decision optimal-selection rate and mean regret are reported below only as *secondary, ecological* statistics describing the full target distribution, and selector performance restricted to discriminative decisions is the **primary** best-candidate-selection metric.

Pairwise preference prediction.. Predict whether candidate A is better than candidate B within the same decision, where “better” means lower `y_loss` or equivalently higher `y_value`. This task uses the pairwise view or shipped pairwise sample as a simpler interface for within-decision preference modeling. Because equal losses induce ties, reports must state whether evaluation includes ties, excludes them, or treats them as a separate class. In the shipped sample, deterministic hash randomization of A/B orientation keeps the non-tie A-better/B-better split balanced (about 49.8%/50.2%) rather than encoding candidate-identifier order.

Candidate-label distribution characterization.. Summarize the empirical label surface – distributions of `y_loss`, `y_value`, regret, and tie frequency across families, capacities, horizons, and splits – without training a predictive model; this descriptive task underpins interpretation of the other three (Table 7).

The empirical scope of this paper remains intentionally narrow. Rather than presenting a large model bake-off, we report lightweight baselines that can be executed directly from the committed release views and that establish reproducible reference points for later work.

The current results surface covers three such references. A linear regression baseline over candidate rows predicts `y_loss` from released features. It is

intentionally weak: test MAE is 3.8739 with RMSE 4.7152 and $R^2 = 0.0026$, and the validation-split R^2 is slightly negative (-0.0506). Using that same linear score for within-decision selection nevertheless provides a reproducible decision-level floor. On the pairwise task, the earlier random and majority non-tie results remain useful sanity checks, but by themselves they do not test whether the released candidate features contain decision-relevant signal.

To test whether released candidate features carry pairwise signal without changing the benchmark schema, we join those features back to the capped shipped pairwise sample for evaluation only. This remains a sample-backed evaluation, not a full quadratic pairwise materialization. The canonical re-generated sample is itself tie-heavy: 878,262 of 1,000,000 rows (87.8%) are exact ties, leaving 121,738 non-ties. All classifier metrics below are therefore non-tie-only.

Fitting and evaluation protocol. Logistic score calibrations and the feature-difference model are fit only on non-tie `train` rows, with fixed L2 regularization and no validation tuning. The temporally held-out `test` split is the generalization result; train/validation columns are diagnostic. On the 2,211-row test non-tie subset, the logistic feature-difference model reaches 0.7078 accuracy (Wilson 95% CI [0.688, 0.726]) and 0.5494 log loss, ahead of random/majority checks and the 1-D score baselines (Table 6). This corrected test non-tie accuracy represents the authoritative baseline for pairwise generalization on the held-out split. The train/validation gap is large, and the test non-tie subset is small and unusually tie-heavy, so the result supports only a capped-sample, non-tie pairwise baseline, not a population or online-policy claim.

This test non-tie subset is also highly workload-concentrated: `twem-cache` and `alibaba-block` together supply 94.98% of it (58.16% and 36.82% respectively, of 2,211 rows), while `metacdn` contributes zero test non-ties because `metacdn` has zero decisions of any kind in the test split at the decision-view level (Section 9) – a full-family train/val/test partitioning property, not an artifact of pairwise sampling. `wiki2018` likewise contributes no test non-ties, consistent with its complete tie-degeneracy. The 0.7078 result therefore demonstrates decision-relevant signal on the available held-out non-tie population, not generalization across all five workload families. We found no evidence of feature leakage: none of the 26 candidate-level feature columns used here reference `y_loss`, `y_value`, `regret`, or any split-identity quantity. Separately, 18 of those 26 – `candidate_predictor_score` and seventeen related predictor-, bucket-, and

confidence-derived fields (discussed in Section 17) – are hard-coded placeholder constants across the entire release, so after standardization their feature difference is exactly zero and they contribute no decision signal to the model. A further feature, `candidate_is_predictor_victim`, is bit-identical to `candidate_is_lru_victim` and is therefore exactly redundant. In total, 19 of the 26 supplied features provide no additional independent variation, leaving seven that are nonconstant and nonredundant; this establishes only nonconstancy, not that those seven carry independent predictive signal. The large train (0.9598) / validation (0.9896) versus test (0.7078) accuracy gap is most consistent with workload/split distribution shift, given the test set’s family concentration described above, rather than a demonstrated leakage or sampling-bug mechanism; we have not established this mechanism conclusively and do not claim more than that here.

The best-candidate evaluator groups candidate rows by the canonical 9-column decision key and reports 2,363,286 decisions, an exact match with the decision view (Section 3). A cheap decision-level floor computed directly from the decision view shows why this match matters for interpretation: a uniform-random selector’s expected optimal-selection rate is 0.9912 with expected regret 0.0088, both more favorable than the linear-score selector’s observed 0.8753 optimal-selection rate and 0.1251 mean regret, purely because of the tie density documented in Section 9 – not because the linear score underperforms a stronger baseline. These all-decision figures are secondary, ecological statistics; headline selection-rate and regret numbers should therefore never be read in isolation from either the random floor or the discriminative-only figures below.

764,282 of 2,363,286 decisions (32.34%) are discriminative, per the definition above. Restricted to this stratum – the only decisions on which a selector’s choice can affect realized regret – the uniform-random selector’s expected optimal-selection rate falls to 0.9729 (expected regret 0.0273), while the linear-score selector’s optimal-selection rate falls to 0.6145 (mean regret 0.3867). These discriminative-only figures follow exactly from the aggregate all-decision totals, because every all-tied decision necessarily contributes optimal-selection = 1 and regret = 0 for any selector. On the decisions where the finite-horizon target actually distinguishes candidates, the linear score is therefore substantially worse than uniform random – not merely comparable to it – and we report this directly rather than only the more favorable all-decision figures. We do not rescue this baseline with an alternative normalization or by excluding difficult cells; Table 4 reports both views side by

Table 4: Candidate-label and best-candidate-selection summaries from validated result records.

statistic	value
candidate rows summarized	277,995,072
y_loss mean / std	7.7561 / 4.6542
y_loss q05 / q25 / q50 / q75 / q95	2 / 4 / 7 / 11 / 16
y_value mean / std	-7.7561 / 4.6542
y_value q05 / q25 / q50 / q75 / q95	-16 / -11 / -7 / -4 / -2
manifest-backed decision rows	2,363,286
best-candidate evaluator decision count	2,363,286 (exact match)
<i>all decisions (secondary, ecological view)</i>	
best-candidate optimal-selection rate	0.8753
best-candidate regret mean / median / max	0.1251 / 0 / 3
random-selector expected optimal-selection rate	0.9912
random-selector expected regret	0.0088
<i>discriminative decisions only (primary selector-quality view)</i>	
discriminative decisions (opt. candidate count < candidate count)	764,282 (32.34%)
best-candidate optimal-selection rate	0.6145
best-candidate mean regret	0.3867
random-selector expected optimal-selection rate	0.9729
random-selector expected regret	0.0273

side.

Table 5: Pairwise-sample-backed summary statistics for the shipped sample view.

statistic	value
pairwise rows	1,000,000
split breakdown	test=102,800; train=738,184; val=159,016
family breakdown	wiki2018=253,072; alibaba-block=243,808; metakv=191,328; twemcache=168,504; metacdn=143,288
unique decisions represented	125,000
label distribution	a_better=60,673; b_better=61,065; tie=878,262
A/B orientation method	deterministic_hash_v1 (randomized, not lexicographic)

Table 6: Feature-based pairwise baselines on non-tie rows only (121,738 of 1,000,000 sampled pairs; 878,262 ties omitted). Models are fit on **train**; **test** is the primary held-out result. Test accuracy 95% Wilson CI: [0.688, 0.726]. The 2,211-row test non-tie subset is 94.98% twemcache/alibaba-block and contains zero metacdn or wiki2018 rows (Section 8), so this result should be read as held-out signal on the available non-tie population, not cross-family generalization.

baseline	feature / model	test ($n=2,211$)		train	val
		acc.	log loss	acc.	acc.
random non-tie	random-label check	0.4903	0.6931	0.5030	0.4971
majority non-tie	majority-class check	0.4853	0.6933	0.5030	0.5000
LRU-score pairwise	1-D logistic, LRU	0.6427	0.9285	0.9141	0.9508
predictor-score pairwise	score				
	1-D logistic, predictor	0.4853	0.6933	0.5030	0.5000
linear-score pairwise	score				
	logistic, linear score	0.6621	0.5512	0.9430	0.9735
logistic feature difference	logistic, feature diffs	0.7078	0.5494	0.9598	0.9896

Table 7: Benchmark task definitions and representative evaluation outputs.

Task	Inputs	Representative outputs
Value regression	candidate rows	MAE, RMSE, and optional rank correlation between predicted and observed values
Best-candidate selection	decision groups	Optimal-selection rate and mean regret on discriminative decisions (optimal-set size smaller than candidate count) as the primary view; all-decision Top-1 accuracy against the benchmark-optimal set reported only as secondary, ecological context given extreme tie density
Pairwise preference	pairwise rows from one decision	Accuracy or ROC-AUC, with tie handling stated explicitly and non-tie-only reporting marked when used
Label characterization	candidate or decision aggregates	Histograms, quantiles, tie rates, regret summaries, and per-family/capacity/horizon breakdowns

9. Target Discriminativeness (RQ1)

This section answers RQ1: is the finite-horizon counterfactual target discriminative between candidates, and when does it degenerate? Source: `analysis/sigmoid_target_discriminativeness_20260913/`, audited over all 277,995,072 candidate rows / 2,363,286 decisions in the canonical dataset (all capacities, all horizons; the full population, not a sample).

9.1. The uncomfortable aggregate facts, stated first

Pooled across every family, capacity, and horizon: the decision-weighted random-optimal probability is 0.9912, the decision-weighted mean random regret is 0.0088, the all-tied fraction is 0.6766, and the unique-winner fraction is exactly 0.0. In plain terms: a uniformly random candidate choice lands in the optimal set 99.1% of the time, over two-thirds of decisions have every candidate exactly tied, and *no audited decision in this dataset has a single, strictly-best candidate victim*. The overall classification, by the audit’s own pre-registered rule, is **B = CONDITIONALLY DISCRIMINATIVE** – not uniformly discriminative, and this is stated here as the paper’s own finding rather than a limitation discovered post hoc. Aggregate degeneracy this large is, by itself, a substantive result about the target’s shape.

9.2. Where the target is informative

The aggregate picture, however, is not the same at every family, capacity, and horizon – consistent with prior large-scale production evidence that cache workloads vary substantially in access pattern and reuse structure even within a single deployment [13], which is why this section reports per-family results rather than only the pooled aggregate above. Figure 3 shows discriminativeness ($1 - \text{all-tied fraction}$) across all five families, four capacities, and three horizons (source data: same audit, per-stratum breakdown, `outputs/phase4_stratified_family_capacity_horizon.csv`, 60 family $\times$ capacity $\times$ horizon cells). Two families, **metacdn** and **twemcache**, are substantially more discriminative than the aggregate figure suggests – at $H = 16$, **metacdn**’s all-tied fraction is 0.0085–0.012 and **twemcache**’s is 0.10–0.24 across its two smallest capacities, an order of magnitude below the aggregate 0.68. **alibaba-block** and **metakv** sit at the tie-heavy end (all-tied fraction 0.66–0.87 at $H = 16$), closer to but still below the degenerate extreme. **wiki2018** is exactly at the degenerate extreme: all-tied fraction 1.0 at every capacity and every horizon tested, with zero reuse events in its scored windows (Section 12 shows this is

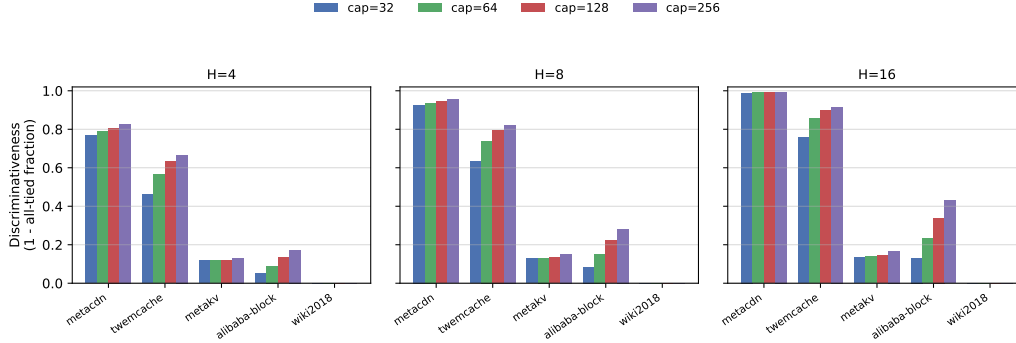


Figure 3: Target discriminativeness (1 – all-tied fraction) by family, capacity, and horizon; every value is computed directly from the frozen validated audit, not hand-entered.

Table 8: Matched-population long-horizon sensitivity. Every row uses the same canonical physical eviction-decision population.

Horizon	Decisions	All-tied	Discrim.	Rand. opt.	E[rand. regret]
16	787,762	0.6193	0.3807	0.9869	0.0132
32	787,762	0.5807	0.4193	0.9797	0.0209
64	787,762	0.5551	0.4449	0.9724	0.0290
128	787,762	0.5434	0.4566	0.9661	0.0361

a mathematical certainty, not a statistical property of the trace, and treats wiki2018 as a genuine negative control rather than a benchmark failure). Discriminativeness also increases with horizon within every non-degenerate family (e.g. twemcache/cap32 rises from 0.46 at $H = 4$ to 0.76 at $H = 16$) – a longer look-ahead window resolves more ties, consistent with more future requests giving more opportunities for candidates’ fates to diverge.

The extended long-horizon comparison asks whether this picture changes when the look-ahead window grows beyond the release horizons. Table 8 and Figure 4 summarize the validated $H \in \{16, 32, 64, 128\}$ comparison. This uses the exact same canonical physical eviction-decision population of 787,762 decisions across all horizons, removing any aggregate denominator mismatch. On this matched population, aggregate discriminativeness increases monotonically with the look-ahead horizon (pooled discriminative fraction rising from 0.3807 at $H = 16$ to 0.4566 at $H = 128$), while the aggregate all-tied fraction decreases monotonically and expected random regret

increases. Thus the changes diminish between H64 and H128, but a saturation claim would be too strong: effects remain heterogeneous by family and capacity. The target also remains strongly tie-degenerate at H128: the decision-micro all-tied fraction is 0.5434.

Look-ahead relative to cache capacity (H/C). To investigate whether the high tie mass is merely an artifact of look-ahead horizons being short relative to cache capacity, we analyze the cell-level H/C ratios across the matched-population dataset, spanning $H/C \in [0.0625, 4.0]$. Substantial degeneracy remains in evaluated cells where H equals or exceeds cache capacity, so the high tie mass cannot be attributed solely to using horizons shorter than the cache: the pooled all-tied fraction is 57.14% at $H/C = 1.0$, 60.45% at $H/C = 2.0$, and 63.83% at $H/C = 4.0$. Because H/C buckets contain different capacity/horizon compositions, these pooled H/C differences should not be interpreted as a causal effect of H/C itself, but rather as evidence that family and capacity composition remain important and that $H \geq C$ regimes are not inherently tie-free.

Additionally, we distinguish between cumulative raw regret and normalized regret per request (regret/ H). On the exact matched 787,762-decision population, raw random regret rises monotonically with H (0.0132 at $H = 16$, to 0.0361 at $H = 128$), but the aggregate normalized regret per request decreases monotonically: 0.000826 at $H = 16$, 0.000653 at $H = 32$, 0.000453 at $H = 64$, and 0.000282 at $H = 128$. For example, for metacdn/cap32 (H/C rising from 0.5 to 4.0), raw random regret rises from 0.0601 ($H = 16$) to 0.0894 ($H = 128$), but the normalized regret per request drops from 0.00375 to 0.00070. Conversely, the most tie-heavy families like alibaba-block and metakv remain consistently low-information and low-regret across all horizons. This indicates that longer look-ahead horizons accumulate more raw opportunity for regret mechanically, but do not increase the per-request target informativeness proportionally.

9.3. Interpretation

The scientific contribution here is partly *characterizing* where the target is informative rather than asserting it is informative everywhere. High tie rates are not hidden or explained away: they are the majority regime in this dataset, and Section 12 connects them directly to trace-level reuse structure (capacity-scale locality), giving a mechanistic account of exactly why metacdn/twemcache are discriminative and alibaba-block/metakv/wiki2018 are not. Sections 10 and 11 then ask whether this same discriminativeness

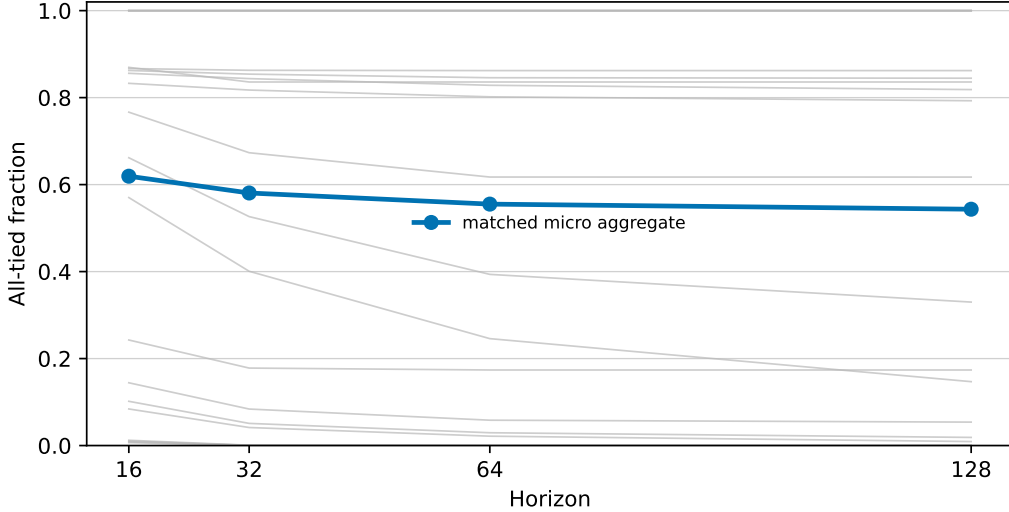


Figure 4: Matched-population long-horizon sensitivity. Thin gray lines are family×capacity cells and the blue line is the decision-micro aggregate; every trajectory follows the same physical eviction decisions across $H \in \{16, 32, 64, 128\}$.

structure predicts anything about actual closed-loop cache-policy behavior (it does, in the regimes identified here).

10. Closed-Loop Policy Evaluation (RQ2, part 1)

Section 9 characterizes the offline target. This section reports the actual closed-loop cache-policy behavior it will be compared against in Section 11 (Table 9). Scope: 5 families $\times$ 2 capacities (32, 128) $\times$ 4 policies (LRU, MRU, random-mean over 20 seeds, SIEVE [3]), per Section 4.

LRU is the best of the four Tier-1 comparators in 9 of 10 cells. The sole exception is metakv/cap128, where SIEVE beats LRU by 4.22% relative. Conversely, **MRU is the worst policy in every non-degenerate cell** by a wide margin. Random-policy seed variability is small (maximum standard deviation 0.0025), which is sufficient to reliably separate wide gaps but means very narrow differences in low-signal cells (like metakv/cap32) fall within the stochastic noise floor. No universal ranking holds for SIEVE vs. random; their relative ordering is strictly family-dependent.

The traces cluster into three regimes. First, **wiki2018 is completely**

Table 9: Closed-loop policy evaluation: LRU miss ratio and relative miss-ratio difference vs. LRU for MRU, mean-random (20 seeds), and SIEVE. metacdn is validation-window evidence; all other families are test-window. Negative values mean the policy beats LRU.

family	cap.	LRU miss ratio	MRU Δ	random Δ	SIEVE Δ	random std
metacdn	32	0.5792	+47.37%	+5.38%	+0.81%	0.0013
metacdn	128	0.5490	+54.93%	+2.98%	+0.36%	0.0014
twemcache	32	0.7511	+27.12%	+4.19%	+8.47%	0.0025
twemcache	128	0.6205	+52.98%	+7.78%	+15.29%	0.0023
metakv	32	0.7410	+10.02%	+0.47%	+0.03%	0.0011
metakv	128	0.7290	+8.57%	−2.07%	−4.22%	0.0015
alibaba-block	32	0.9836	+1.65%	+0.28%	+0.20%	0.0010
alibaba-block	128	0.9561	+4.48%	+0.85%	+1.44%	0.0014
wiki2018	32	1.0000	0.00%	0.00%	0.00%	0.0000
wiki2018	128	1.0000	0.00%	0.00%	0.00%	0.0000

degenerate, with a 1.0 miss ratio and exactly zero standard deviation for all policies, following deductively from its lack of reuse (Section 12). Second, **alibaba-block is a near-tie regime** where all policy gaps remain small. Third, the highly discriminative families (metacdn and twemcache) show massive separation between LRU and MRU. The metakv/cap128 cell remains the only genuine exception to LRU’s dominance, where both SIEVE and random outperform it. As noted in Section 3, metacdn’s results derive from validation-window evidence, while the rest are test-window.

Expanded comparator robustness.. Table 9 compares four policies; to check that its conclusions are not an artifact of that narrow set, we re-ran the identical Tier-1 protocol with three additional deterministic comparators – ARC [1], LIRS [2], and S3-FIFO [4] (full results: Table A.16). Under the expanded seven-policy set, LRU is strictly best in 4 of the 10 cells rather than 9 of 10, but five of the six cells in which another policy now leads LRU do so by less than 3.2% relative (0.45%, 0.91%, 1.29%, 1.68%, 3.19%) – comparable to gaps this paper already treats as near-ties – so most of the reordering is fine-grained jockeying among already-close policies. The substantive exception remains metakv/cap128, where S3-FIFO now beats LRU by 8.07% relative (versus SIEVE’s 4.22% under the original four). Importantly for Section 11, the association between offline discriminativeness and closed-loop policy spread (max–min miss ratio across the compared policies) is unchanged at $r = 0.96$ under either the four-policy or the seven-policy spread definition ($n = 10$ family $\times$ capacity cells, exploratory), so the ob-

served association between offline discriminativeness and closed-loop policy separation is not an artifact of the original comparator set. W-TinyLFU was preregistered for this expansion but excluded after its reference implementation proved non-functional at capacity 32 under its own default configuration (`COMPARATOR_PROTOCOL.md`); it contributes no results here.

11. Offline↔Closed-Loop Correspondence (RQ2, part 2)

This section answers the second half of RQ2 – does the offline candidate ranking from Section 9 correspond to the closed-loop behavior in Section 10? – and, jointly with the horizon comparison below, RQ3’s horizon question. Source: `analysis/closed_loop_offline_linkage_20260914/`, computed with two independent code paths in numerical agreement. **SIEVE has no offline counterpart** (Section 5 defines labels only for the continuation-policy family LRU/MRU/random) and is therefore excluded from every comparison in this section; only MRU-vs-LRU and random-vs-LRU are compared.

RQ-CL1 (rank agreement). At $H = 16$, across the 10 family×capacity cells: MRU-vs-LRU agrees in sign (offline regret-gap sign matches closed-loop miss-ratio-gap sign) in 7 of 10 cells, disagrees in 1, and is tied on both sides (wiki2018) in 2; random-vs-LRU agrees in 6, disagrees in 2, ties in 2. The discordant cells are always the same two: **alibaba-block/cap32** (discordant for both pairs – offline regret gaps there are -0.0027 and -0.0007 , i.e. barely non-zero on the offline side too) and **metakv/cap128** (discordant for random-vs-LRU only – offline says random is very slightly worse than LRU, $+0.0013$ regret gap, while closed-loop shows random modestly beating LRU, -0.015 miss-ratio gap; Section 10 additionally notes SIEVE also beats LRU there). Excluding wiki2018’s two trivially-tied cells, concordance among the 8 non-degenerate cells is not manufactured by wiki2018’s automatic agreement: dropping wiki2018 still leaves the same two discordant cells and the same qualitative picture.

RQ-CL2 (gap-magnitude correspondence). At $H = 16$: MRU-vs-LRU Pearson $r = 0.835$ (Spearman $\rho = 0.915$, Kendall $\tau = 0.773$); random-vs-LRU $r = 0.755$ ($\rho = 0.829$, $\tau = 0.591$). Figure 5 shows both relationships directly, with the two discordant cells annotated rather than hidden. These correlations rest on only 8–10 family×capacity cells and are reported with that n attached everywhere; they are not treated as population-scale estimates.

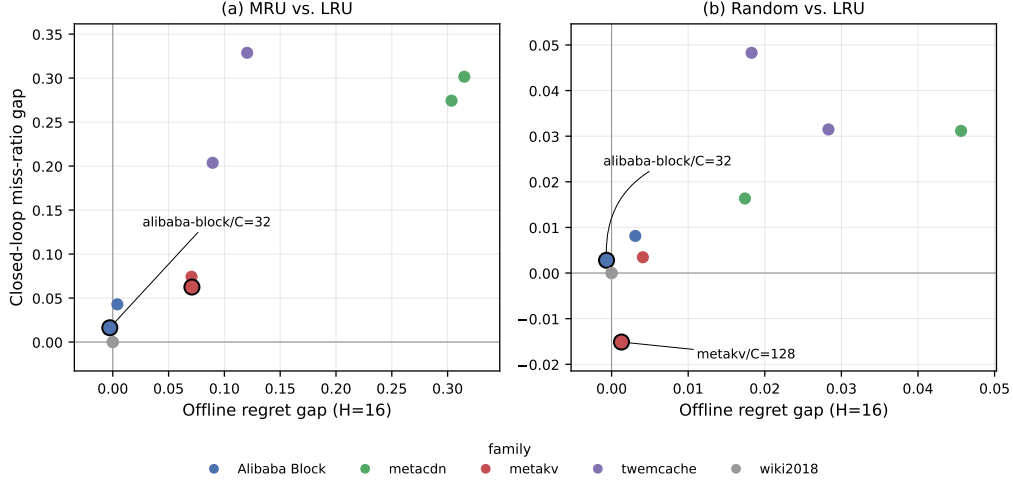


Figure 5: Offline regret gap vs. closed-loop miss-ratio gap at $H = 16$, split into two panels: (a) MRU-vs-LRU and (b) random-vs-LRU, each $n = 10$ family $\times$ capacity cells (one point per cell per panel). **The two panels use different axis ranges**, visible directly from their tick labels: random-vs-LRU gaps (panel b) are uniformly much smaller in magnitude than MRU-vs-LRU gaps (panel a), so each panel is scaled to its own range rather than a shared one. The two discordant cells from RQ-CL1 are outlined and labeled; every other point is left unlabeled.

RQ-CL3 (workload/capacity dependence). metacdn and twemcache are the strong-agreement, high-discriminativeness regimes and also show the largest closed-loop policy separation; alibaba-block has a near-zero offline gap whose sign is not robust; metakv/cap128 is the one reversal; wiki2018 is degenerate on both sides. Across the 10 cells at $H = 16$, an exploratory correlation between offline discriminativeness ($1 - \text{all-tied fraction}$) and closed-loop $|\text{MRU} - \text{LRU}|$ separation gives Pearson $r = 0.968$; Section 12 traces this to shared trace locality.

RQ-CL4 (horizon). $H = 16$ wins or ties on every magnitude-sensitive statistic across every sensitivity slice tested (primary, excluding wiki2018, test-window-only), for both pairs. The coarse concordant/ discordant/tied counts are *identical* at $H = 4$, $H = 8$, and $H = 16$ (the same two cells are discordant regardless of horizon); horizon changes the *strength* of the correspondence (the correlation coefficients above), not which cells agree in sign. The differences between horizons are modest in absolute terms ($r = 0.746$ at $H = 4$ to $r = 0.835$ at $H = 16$ for MRU-vs-LRU) and $n = 8$ –10 per

horizon is small, but the pattern is consistent and not cherry-picked.

Family-aware robustness. The headline statistics above are computed over $n = 10$ cells ($5 \text{ families} \times 2 \text{ capacities}$). Because capacities within the same family are not independent workload replicates, we conducted rigorous family-aware sensitivity checks at $H = 16$. The observed association is stable under leave-one-family-out (LOFO) sensitivity: excluding both cells of any single family leaves the MRU-vs-LRU Pearson correlation positive and strong, ranging from $r = 0.794$ (when wiki2018 is excluded) to $r = 0.993$ (when twemcache is excluded). The random-vs-LRU correlation remains similarly stable, ranging from $r = 0.716$ (wiki2018 excluded) to $r = 0.867$ (twemcache excluded). Furthermore, excluding the metacdn family entirely (the only family evaluated on a validation window rather than test window in closed loop) yields $r = 0.885$ (MRU-vs-LRU) and $r = 0.833$ (random-vs-LRU), indicating the association is not an artifact of that specific window choice. Finally, we evaluate an exact family-level block permutation test under the null hypothesis that the mapping between offline family blocks and closed-loop family blocks is exchangeable while preserving the two capacities within each family block. Under this exact family-block reassignment null (yielding $5! = 120$ possible permutations), the observed MRU-vs-LRU alignment is more extreme than all other block assignments (two-sided $|r|$ $p = 0.0083$), and the Random-vs-LRU result has $p = 0.025$. These exact permutation results quantify robustness within the evaluated family structure; they do not establish population-level generalization beyond the five trace families, as the limited number of trace clusters ($n_{\text{clusters}} = 5$) bounds broad inferential generalization.

Answer to RQ2/RQ3. Offline and closed-loop policy orderings agree in the large majority of workload/capacity regimes where the offline signal itself is not near zero, with two specific, characterized exceptions; $H = 16$ gives the strongest, though modest, alignment among the three horizons tested (Section 17 states the horizon scopes: primary release $H \in \{4, 8, 16\}$, with separate sensitivity reanalysis extending through $H = 128$). This is a bounded, evidenced claim, not a universal predictive-validity claim for the offline labels.

12. Mechanistic Workload Analysis (RQ3, part 2)

Sections 9 and 11 show *that* discriminativeness and offline $\leftrightarrow$ closed-loop agreement vary by workload; this section asks *why*, using only existing trace-

only and closed-loop evidence (no new simulation). Classifications follow the source report’s own discipline exactly: DIRECTLY OBSERVED (a deductive or exactly-reproduced fact), CONSISTENT WITH MECHANISM (plausible, structurally supported, not proven), and CAUSALLY ESTABLISHED (reserved for one deductive result below). No claim here is upgraded beyond its source classification.

Capacity-scale reuse correlates with both the all-tied fraction and closed-loop separation. Trace-derived LRU hit rate (an exact, capacity-scale reuse measure, computed independently of the labels) is negatively associated with the offline all-tied fraction at $r \approx -0.83$ (equivalently positively associated with discriminativeness at $r \approx +0.83$) and positively associated with closed-loop $|\text{MRU} - \text{LRU}|$ separation at $r \approx 0.90$ (pooled $n = 10$). Figure 6 shows both relationships directly. The two associations are not equally robust to workload composition: leave-one-family-out sensitivity analysis leaves the separation correlation comparatively stable, but reduces the discriminativeness correlation from $r \approx 0.83$ to $r \approx 0.69$ when metacdn is excluded (exact family-level permutation $p \approx 0.117$), so we treat the discriminativeness pairing as consistent with, rather than robust confirmation of, the proposed mechanism. This is an $n = 10$, exploratory finding, consistent with but not independent evidence from RQ-CL3’s own exploratory correlation (Section 11) – the same underlying trace property is driving both.

alibaba-block/cap32 is near-zero signal. Only 17.5% of window requests are reuses at all; of those, only 9.4% fall within stack distance 32. The trace-only predicted LRU hit rate ($0.175 \times 0.094 = 1.6\%$) exactly reproduces the observed Tier-1 LRU hit rate ($134/8192 = 1.64\%$). The offline regret gaps there (-0.0027 , -0.0007) are tiny on a scale where metacdn’s own MRU-LRU gap is over $100\times$ larger. The Section 11 discordance at this cell is exactly what a coin-flip-scale signal would produce.

metakv/cap128 remains a bounded exception. metakv’s reuse-distance median is short (2, comparable to metacdn’s 3) but its tail is markedly heavier: reuse-within-32 is only 0.574, rising to just 0.600 by distance 128 – a narrow, 2.6-percentage-point “boundary zone” sitting almost exactly at the tested capacities, where a recency-strict policy (LRU) and a second-chance policy (SIEVE) can plausibly diverge. metakv’s cold-start scoring window (Section 4) is a second, independently real structural property of this same cell. **Neither is shown to be the cause:** no per-request SIEVE or random trajectory log exists in the validated evidence to isolate which property (or their interaction, or neither) produced the reversal, and

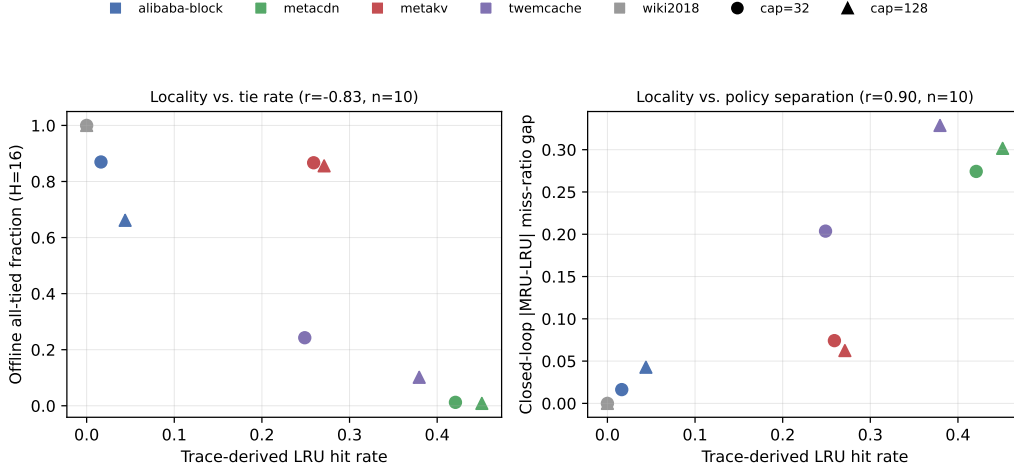


Figure 6: Trace-derived LRU hit rate vs. offline all-tied fraction (left) and closed-loop policy separation (right), $n = 10$ family $\times$ capacity cells.

the source analysis states this explicitly rather than forcing a resolution.

twemcache’s random-over-SIEVE ordering is not explained by existing data. twemcache has the strongest “small hot set plus long tail” bi-modal signature in the dataset (longest novel streak 3; reuse distance p10=3 but p75=7379), which would naively predict SIEVE’s second-chance admission should help – but the observed result is the opposite (SIEVE loses to random at both capacities). Existing trace-only data can describe this structure but cannot explain the direction of the outcome; the source analysis classifies any “SIEVE benefits from bimodality” story here as NOT SUPPORTED by the data, and this paper does not repeat that story.

wiki2018 (causally established): the one exception to the paper’s general classification discipline, and deliberately so. Zero reuse events occur in the scored window ($n_{\text{reuse}} = 0$ across all 4096 requests, every one a first-ever occurrence over the full 50,000-request trace). Since a hit requires a repeated reference to an already-resident item, miss ratio = 1.0 for every policy is a mathematical certainty given zero reuse events, not a statistical tendency – an existence proof from a directly counted quantity, which is why this one result is classified CAUSALLY ESTABLISHED while every other mechanism in this section is not.

13. Continuation-Policy Sensitivity (RQ4)

The canonical labels use LRU as the continuation policy after a forced eviction. RQ4 asks whether the optimal-candidate set and regret ordering are materially changed if followed by a different continuation policy. Table 10 summarizes this sensitivity evidence.

Sampled MRU/random study. A pre-registered sampled study evaluates 5,500 decisions at capacities 32 and 128. By the pre-registered informative-stratum (Set C) rule, both MRU and mean-random continuation classify ROBUST. MRU is very close to LRU: Set-C mean Jaccard is 0.9958 (cap 32) and 0.9999 (cap 128). Mean-random continuation is more perturbing but still robust: mean Jaccard is 0.7251 (cap 32) and 0.8522 (cap 128).

Full-population MRU census. The full-population MRU census extends the MRU result to all five families and capacities 32, 64, 128, and 256. Across all 2,363,286 decision-horizon units, mean optimal-set Jaccard is 0.99941, median Jaccard is 1.0, and 98.69% of records have exact Jaccard 1. Strict preference reversals are extremely rare (1.87×10^{-7}).

Because 67.31% of records are all-tied under both LRU and MRU, the pre-registered decisive stratum is Set C (764,282 records). Here, mean Jaccard is 0.99846, median Jaccard is 1.0, and mean cross-continuation regret is 0.000638 misses. Since wiki2018 contributes no Set-C records, this is not inflated by the degenerate negative control.

Scope and limitations. Sensitivity is largest at small capacities and long horizons, but remains small in absolute terms. The worst Set-C cell (alibaba-block/cap32/ $H = 16$) yields mean Jaccard 0.9804 and median 1.0, satisfying the pre-registered robustness criterion. These results substantiate robustness within the evaluated scope (sampled random at capacities 32/128, and full-population MRU). They do not establish population-scale random-continuation robustness or guarantee that arbitrary deployed policies would preserve the ordering.

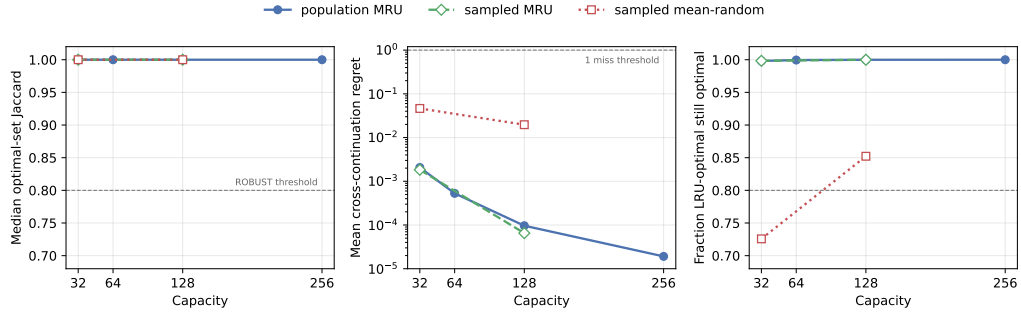


Figure 7: Continuation-policy sensitivity in the pre-registered informative stratum (Set C). Sampled MRU and mean-random evidence covers capacities 32 and 128 only; full-population MRU evidence covers capacities 32–256.

Table 10: Offline $\leftrightarrow$ closed-loop correspondence and continuation-policy robustness, summarized. The continuation rows use Set C (discriminative-under-LRU). Sampled random evidence is available only at capacities 32/128; full-population evidence is MRU-only.

<i>Offline$\leftrightarrow$closed-loop, $H=16$ ($n=10$ cells; SIEVE excluded)</i>		
metric	MRU-vs-LRU	random-vs-LRU
Concordant / discordant / tied (of 10)	7/1/2	6/2/2
Pearson r	0.835	0.755
Spearman ρ	0.915	0.829
Kendall τ -b	0.773	0.591
<i>Sampled continuation robustness, Set C, by capacity</i>		
metric	cap. 32	cap. 128
MRU median / mean Jaccard	1.000 / 0.9958	1.000 / 0.9999
MRU mean cross-cont. regret	0.0018	0.0001
MRU strict reversal fraction	2.7×10^{-5}	0.0
random median / mean Jaccard	1.000 / 0.7251	1.000 / 0.8522
random mean cross-cont. regret	0.0463	0.0197
random strict reversal fraction	1.0×10^{-3}	1.5×10^{-6}
<i>Full-population MRU census, Set C</i>		
metric	all capacities	new capacities 64/256
Scope	764,282 decision-H units	391,499 decision-H units
median / mean Jaccard	1.000 / 0.998459	1.000 / 0.999425
mean cross-cont. regret	0.000638	0.000267
strict reversal fraction	5.38×10^{-7}	2.83×10^{-7}
LRU-opt. still MRU-opt. fraction	0.999396	0.999745
robustness classification	ROBUST	ROBUST

14. Practical Implications (RQ5)

This section synthesizes the preceding results into guidance for using finite-horizon counterfactual supervision.

When should the offline signal be trusted? Most directly in workload/capacity regimes with high capacity-scale reuse, identifiable before closed-loop replay via a high trace-derived LRU hit rate or low all-tied fraction. In this dataset, this corresponds to metacdn- and twemcache-like regimes, where offline discriminativeness and closed-loop agreement are strongest.

When is caution warranted? In near-zero-signal regimes (e.g., alibaba-block/cap32), small offline regret gaps may flip signs without reflecting genuine disagreement. Caution is also needed when interpreting cold-start evaluation windows (metakv) or validation-only regimes (metacdn). A high tie rate is itself informative: it identifies where policy comparison is fundamentally unresolvable by this label family (e.g., wiki2018), preventing researchers from hunting for non-existent signal.

Why still run closed-loop evaluation? The offline/closed-loop association is real but bounded, resting on $n = 10$ cells with at least one unexplained exception (metakv/cap128). Offline-label results are complementary diagnostics and cannot replace closed-loop performance claims.

How should continuation-policy assumptions be interpreted? As a tested robustness property with real limits. The labels are highly robust to MRU continuation at population scale, and acceptably robust to random continuation in the validated sample. This supports the use of canonical LRU-continuation labels in evaluated regimes, but does not imply continuation choice is universally irrelevant.

15. A Frozen Learned-Policy Closed-Loop Check

Section 10 evaluates non-learned policies against the Tier-1 production closed-loop protocol. This section reports a separate, smaller confirmatory experiment: freezing one model trained on this benchmark’s supervision and evaluating it, in closed loop, against matched baselines under its own held-out test protocol. **This experiment uses an independently constructed held-out test split (the frozen model’s own training-pipeline split, applied identically across all five families, including metacdn) and is not run on the Tier-1 protocol; its miss-ratio values are therefore not numerically comparable, cell for cell, to Table 9.** What is

comparable, and is the point of this section, is the ordering among policies evaluated together under one matched, held-out protocol.

Setup. We refer to this baseline model as HGB (scikit-learn’s `HistGradientBoostingRegressor`). It was trained to predict the $H = 16$ `y_loss` target, selected by offline validation, and frozen. As a pre-repair artifact, its serialized schema reflects the legacy 26-feature interface. A forensic audit deprecated 19 of those columns (18 constant, 1 redundant alias); structural checks verified the fitted estimator never branches on the constants, though it uses the redundant alias (carrying no independent information). Consequently, HGB is retained as a historical baseline rather than a clean seven-feature model. The policy was evaluated by true closed-loop replay – evicting the candidate with the lowest predicted `y_loss` – on the same 10 cells against matched LRU, MRU, SIEVE, LFU, and random baselines under identical warmup and scoring conventions.

Table 11: Frozen learned-policy closed-loop check: win/tie/loss count for the learned policy against each comparator across the same 10 predetermined cells (5 families $\times$ capacities 32, 128), under a held-out protocol distinct from Table 9. A tie is exact floating-point equality; every observed tie is one of the two wiki2018 cells, where all 6 policies score miss ratio 1.0. Macro miss ratio is the unweighted mean over the 10 cells (lower is better).

comparator	learned wins	ties	learned losses	macro miss ratio (comparator)
LRU	1	2	7	0.7117
MRU	8	2	0	0.8484
SIEVE	1	2	7	0.7096
LFU	5	2	3	0.7825
random (20-seed mean)	1	2	7	0.7228
learned macro miss ratio				0.7629

Result. Table 11 shows the frozen policy did *not* outperform strong heuristics. Outside the two fully degenerate wiki2018 cells, the learned policy beat LRU, SIEVE, and the random-seed mean in exactly one cell (metakv/cap128) and lost in the other 7. It beat MRU in every non-degenerate cell and beat LFU mostly (5 wins, 3 losses). In the 7 loss cells, the gap is often large: at twemcache/cap128, the learned policy (miss ratio 0.3909) loses to every single one of the 20 random seeds (range 0.1586–0.1709).

Reading this result. This shows that, for this one frozen model, good offline candidate prediction does not by itself imply competitive closed-loop cache performance. The offline surrogate evaluates each decision against a

fixed history, while closed-loop outcome is cumulative and path-dependent. Poorly bounded state divergence, action-underdetermined labels, continuation-policy mismatch, and this predict-then-act distribution shift are all consistent with the observed gap. None is established as the sole cause by this evidence alone, but the result empirically validates the benchmark’s central thesis: offline labels complement, but do not replace, closed-loop validation.

16. Discussion

What LAFC-Evict enables. It provides a reusable, candidate-level, finite-horizon counterfactual supervision surface. This allows researchers to explore value regression, best-candidate selection, and pairwise preference learning without repeatedly re-deriving labels from raw traces. As positioned in Section 18, this public scale and granularity are distinct from existing benchmarking efforts.

What the evaluation reveals. Four critical insights emerge from evaluating the surface against closed-loop behavior. First, target discriminativeness is highly workload-structured, not incidental noise (Section 9). Second, this structure is mechanically tied to capacity-scale reuse (Section 12). Third, offline policy orderings successfully predict closed-loop performance in most informative regimes, though notable exceptions exist (Section 11). Fourth, the canonical LRU-continuation assumption is robust within the evaluated capacity bounds (Section 13).

The role of degenerate regimes. Cases like wiki2018’s complete degeneracy and alibaba-block’s near-zero signal are fully explained results, not anomalies. Understanding exactly why and where the target degenerates makes the non-degenerate claims trustworthy and provides a crucial diagnostic tool for researchers.

Offline surrogate vs. closed-loop validation. This paper’s central position is that offline counterfactual supervision is complementary to, not a replacement for, closed-loop evaluation. As our frozen learned-policy check demonstrated (Section 15), offline predictive structure does not automatically yield closed-loop cache-miss gains, because predict-then-act path dependence fundamentally alters future trajectories. LAFC-Evict therefore explicitly embraces offline-to-online correspondence testing as a necessary, integrated part of evaluating learned eviction strategies.

17. Limitations, Ethics, and Reuse

The evaluations and claims in this paper are bounded by several explicit constraints:

- **Target Scope:** The label is a finite-horizon counterfactual miss cost under a specified continuation policy, not an offline-optimal label for the full future control problem. The canonical release uses $H \in \{4, 8, 16\}$; the matched extended sensitivity check ($H \in \{16, 32, 64, 128\}$) uses a separate simulation pool that does not alter the public release.
- **Continuation Policy:** Canonical labels assume LRU continuation. The sampled and full-population studies establish robustness only within their tested limits (MRU and mean-random); arbitrary continuation policies remain outside this claim.
- **Cache Abstraction:** The evaluated dataset assumes unit-object caching, count-based capacities (32–256), and unconditional admission. Variable sizes, byte-weighted costs, and admission-gated models are not covered.
- **Target Degeneracy:** The target is substantially tied (aggregate all-tied fraction ≈ 0.68), heavily influenced by trace-level reuse structures (Section 9).
- **Family Caveats:** As noted in Section 3, metacdn’s closed-loop evidence relies on a validation-window split, and metakv is evaluated from a cold start.
- **Feature Schema:** A forensic audit found 19 of the 26 historical candidate-level feature columns to be constant or redundant; only 7 carry independent nonconstant variation. No claim relies on the deprecated fields being informative.
- **Governance and Provenance:** Reuse must preserve the attribution and licensing constraints of upstream traces. While all five evaluated families are publicly available in the Hugging Face v1.0 release, hosting remains asymmetric across platforms: AWS remains v0.3/wiki2018-only and Zenodo remains v0.2.

- **Learned-Policy Bounds:** The frozen learned-policy check (Section 15) covers only one model class, target, and feature set. It demonstrates offline-to-online divergence for that specific configuration, but does not serve as an exhaustive study of learned architectures or represent a ceiling on learned-eviction performance.

18. Related Work

18.1. Caching Infrastructure and Benchmarks

Cache studies rely on realistic traces and reusable tooling. The UMass Trace Repository and production workload studies provide traces for evaluation [14, 15, 13]. CacheLib, libCacheSim, and `cache_dataset` provide cache-engine simulation [16, 17, 18], while broader systems artifacts like YCSB and Park show the value of reusable learning-augmented evaluation infrastructure [19, 20, 21, 22].

18.2. Learned and Adaptive Replacement

Recent learned-caching work typically contributes a policy, policy-selection scheme, or learning framework. LRB, LHD, GL-Cache, and 3L-Cache focus on low-overhead or feature-rich per-object eviction [7, 23, 9, 11]. LeCaR, CACHEUS, MAT, and ALPS form another cluster of adaptive policies [6, 10, 24, 25], while HR-Cache, Parrot, Hawkeye, and Mockingjay further explore eviction decisions [26, 8, 27, 28]. Baleen addresses the adjacent admission/prefetching axis [29].

18.3. Heuristics and Learning-Augmented Theory

Classical and modern heuristics define the landscape LAFC-Evict interfaces with, ranging from recency/frequency-aware policies (LIRS, ARC, CLOCK-Pro, GreedyDual-Size [2, 1, 30, 31]) to lightweight admission-aware designs (TinyLFU, S3-FIFO, SIEVE, LAH [32, 4, 3, 5]). Learning-augmented caching theory studies algorithms with predictions and robustness guarantees [33, 34, 35, 36, 37]. LAFC-Evict does not supply algorithmic guarantees, but asks how reusable per-decision supervision can be constructed and characterized. The subsampling study of Wang et al. identifies extremely long decision horizons as a practical obstacle for reinforcement learning [38], motivating our focus on finite-horizon supervision.

18.4. Offline versus Deployed Evaluation

Offline and deployed behavior need not agree when actions affect future observations. DAgger formalizes this distribution-shift problem for imitation learning [39], and similar gaps are documented in recommender systems [40] and caching throughput optimizations [41]. This paper asks a specific question in this space: whether finite-horizon counterfactual miss-cost gaps align with closed-loop miss-ratio gaps for cache eviction.

18.5. Positioning LAFC-Evict

Given this literature, LAFC-Evict’s contribution is a public cache-eviction artifact that releases reusable candidate-level finite-horizon counterfactual supervision, together with decision-aligned candidate, decision, and pairwise task views, at production scale and decoupled from any particular downstream learned method. To clarify LAFC-Evict’s novelty boundaries relative to the closest prior works, we distinguish it on several axes:

- **Chłedowski et al.** [35] provide an experimental study of robust learning-augmented caching and release valuable workload/data resources. LAFC-Evict differs in artifact granularity and intended use: rather than evaluating online policies with predictions under theoretical bounds, it materializes reusable candidate-level finite-horizon counterfactual miss costs plus derived decision/pairwise views, and systematically evaluates their correspondence with closed-loop behavior.
- **Cache-Coliseum** [42] provides a comprehensive algorithm and execution benchmark for evaluating learning-augmented caching policies. In contrast, LAFC-Evict is a data-centric supervision benchmark providing static, pre-computed counterfactual labels. The two resources benchmark different objects: Cache-Coliseum benchmarks policy execution frameworks, while LAFC-Evict packages the standardized label surface those policies can train or be audited against.
- **HALP** [12] leverages preference-based learning for eviction, where preference signals are generated and consumed online during policy training and deployment. In contrast, LAFC-Evict provides a static, materialized offline dataset of counterfactual y_{loss} labels and a deterministic pairwise preference view derived directly from it, reusable independently of a particular deployed learned policy.

- **Belady’s look-ahead optimum** [43] represents the theoretical full-future offline-optimal policy. In contrast, LAFC-Evict’s primary supervision target (y_{loss}) represents the candidate-level finite-horizon counterfactual miss cost under a specified continuation policy. This distinction is conceptually crucial, as the local finite-horizon “optimal set” under a continuation policy is not equivalent to Belady’s full-future global optimal choice.

Ultimately, LAFC-Evict does not claim to propose a new online eviction policy, nor to be the first use of future-access supervision, preference learning, or benchmarking in this space. Rather, its novelty lies in providing a public, reusable, candidate-level finite-horizon counterfactual supervision artifact at production scale; supplying aligned candidate, decision, and pairwise views; systematically characterizing when the target is informative; conducting continuation-policy robustness analysis; and providing a direct empirical comparison against closed-loop policy behavior.

19. Conclusion

This paper positions LAFC-Evict as a benchmark for learned cache eviction, releasing candidate-level counterfactual supervision alongside derived decision and pairwise tasks. The open release enables researchers to train and evaluate policies against finite-horizon miss costs at scale, without requiring repetitive derivation of labels from raw traces.

We characterize this target empirically, revealing that it is frequently tie-degenerate but predictably discriminative based on trace-level reuse structures. Crucially, in regimes where the offline signal separates candidates, offline policy rankings correspond positively with true closed-loop miss-ratio performance. This correspondence is robust to variations in the assumed continuation policy (tested against MRU and random alternatives) and family-aware statistical bounds.

However, offline performance does not guarantee closed-loop success. As demonstrated by a frozen learned-policy check, predicting the offline target well does not automatically yield closed-loop cache-miss reductions over robust heuristics like LRU and SIEVE. LAFC-Evict explicitly embraces this mismatch, emphasizing that static offline supervision complements, rather than replaces, closed-loop evaluation. Future work should explore on-policy training, trajectory-aware objectives, and richer architectures designed to robustly bridge this predict-then-act distribution shift.

Acknowledgments

The author gratefully acknowledges Prof. Ioannis (Yiannis) Koutis for his guidance and support. Computationally intensive experiments used the Wulver high-performance computing system at the New Jersey Institute of Technology. The author also acknowledges support from the Google Cloud Research Credits Program; Cohere Labs through the Catalyst Grant Program, which provided \$1,000 in API credits; and CloudRift AI through the AI Builder Grant, which provided 1,000 credits. The author further thanks Amazon Web Services, through the AWS Open Data Sponsorship Program, for sponsored hosting of up to 600 GB for two years to support public access to LAFC-Evict.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The LAFC-Evict repository is public at <https://github.com/SoroushVahidi/lafc-evict-dataset>. Code in the repository is released under the MIT License where currently documented. The full five-family LAFC-Evict v1.0 public release is available on Hugging Face (<https://huggingface.co/datasets/SoroushVahidi/lafc-evict>) under the v1.0 revision, and packages all five evaluated families: alibaba-block, metacdn, metakv, twemcache, and wiki2018. It contains 277,995,072 candidate-level rows, 2,363,286 decision-horizon rows, and a 1,000,000-row canonical pairwise sample. The dataset is multi-licensed by family: wiki2018 is released under CC0-1.0; twemcache and alibaba-block under CC BY 4.0; and metakv and metacdn under the Apache License, Version 2.0. The older v0.3 wiki2018-only release remains separately available on the Hugging Face `main` revision and AWS Open Data (bucket `lafc-evict-open-data`, region `us-west-2`); AWS does not yet host the five-family v1.0 release. Zenodo currently archives v0.2 only (version DOI 10.5281/zenodo.21895844; concept DOI 10.5281/zenodo.21895843); no v1.0 Zenodo version DOI has been minted.

Funding

This work received computing/API support through the Google Cloud Research Credits Program, the Cohere Labs Catalyst Grant Program (\$1,000 in API credits), and the CloudRift AI Builder Grant (1,000 credits). Public hosting of the LAFC-Evict dataset is supported by the AWS Open Data Sponsorship Program, which provides sponsored hosting of up to 600 GB for two years.

CRedit authorship contribution statement

Soroush Vahidi: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Appendix A. Additional Release Tables

This appendix carries release/schema-descriptive tables that support reproducibility but do not themselves answer a research question posed in the main text. Content is unchanged from the underlying release manifest; nothing here was recomputed for this appendix.

Table A.12: Dataset scale from preserved-release metadata. Counts are manifest-backed or view-backed as noted.

metric	value
candidate rows	277,995,072
decision rows	2,363,286
pairwise-sample rows	1,000,000
preserved release size	2.7G
candidate parquet files	168
total release files	176
manifest file_inventory count	176
checksum entry count	175
selected trace families	alibaba-block, metacdn, metakv, twemcache, wiki2018
excluded trace families	brightkite, citibike
full validation status	passed

Table A.13: Split counts from manifest-backed candidate-row metadata. These are candidate-row counts, not decision-view counts.

split	row_count
test	28,205,952
train	205,726,368
val	44,062,752

Table A.14: Decision-view-backed benchmark composition by family, capacity, horizon, and split.

dimension	counts
trace_family	wiki2018=598,560; alibaba-block=576,219; metakv=454,053; twemcache=397,494; metacdn=336,960
capacity	32=613,314; 64=599,172; 128=582,678; 256=568,122
horizon	4=787,762; 8=787,762; 16=787,762
split	test=242,418; train=1,746,882; val=373,986

Table A.15: Decision-view-backed candidate-count statistics. These values summarize the number of candidates per decision in the preserved release.

group	min	med	mean	max
overall	32	64	117.63	256
alibaba-block	32	64	118.35	256
metacdn	32	64	117.12	256
metakv	32	64	119.52	256
twemcache	32	64	111.52	256
wiki2018	32	64	119.85	256

Table A.16: Expanded Tier-1 closed-loop comparison (Problem 5): miss ratio by family, capacity, and policy. LRU/MRU/random/SIEVE are reproduced byte-for-byte from the frozen Tier-1 evidence of Table 9; ARC [1], LIRS [2], and S3-FIFO [4] were added under the identical protocol (same scored windows, trace hashes, and simulator commit; deterministic reference implementations, no trace-specific tuning). W-TinyLFU was preregistered but excluded after its reference implementation proved non-functional at capacity 32 under its own default configuration (see `COMPARATOR_PROTOCOL.md`). Source: [analysis/problem5_expanded_closed_loop_20260917/](#).

Family	Cap.	LRU	MRU	Random	SIEVE	ARC	LIRS	S3-FIFO
metacdn	32	0.5792	0.8535	0.6103	0.5838	0.5766	0.6082	0.5983
metacdn	128	0.5490	0.8506	0.5654	0.5510	0.5440	0.5890	0.5576
twemcache	32	0.7511	0.9548	0.7826	0.8147	0.7545	0.8745	0.8549
twemcache	128	0.6205	0.9492	0.6688	0.7153	0.6241	0.8435	0.7687
metakv	32	0.7410	0.8152	0.7444	0.7412	0.7432	0.7952	0.7285
metakv	128	0.7290	0.7915	0.7139	0.6982	0.6865	0.7202	0.6702
alibaba-block	32	0.9836	0.9999	0.9864	0.9856	0.9740	0.9719	0.9709
alibaba-block	128	0.9561	0.9989	0.9642	0.9698	0.9274	0.9255	0.9325
wiki2018	32	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
wiki2018	128	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During manuscript preparation, the author used ChatGPT (OpenAI), Gemini (Google), Claude (Anthropic), and Perplexity AI for drafting and editing support, research organization, and literature/web exploration. After using these tools, the author reviewed and edited the content and takes full responsibility for the published article. No AI tool is listed as an author or contributor.

AI-assisted code and research workflows. ChatGPT, OpenAI Codex, Claude, Cohere services/models, and CloudRift-hosted AI models were also used where applicable for code development, debugging, experiment organization, and research workflows. Exact Cohere and CloudRift model identifiers were not preserved and are therefore not invented here. All scientific claims, code, tables, and figures were reviewed by the author and trace to validated repository artifacts rather than to AI output alone.

References

- [1] N. Megiddo, D. S. Modha, Arc: A self-tuning, low overhead replacement cache, in: Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST), 2003, pp. 115–130.
URL <https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache>
- [2] S. Jiang, X. Zhang, Lirs: An efficient low inter-reference recency set replacement policy to improve buffer cache performance, in: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2002, pp. 31–42. doi:10.1145/511334.511340.
- [3] Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, K. V. Rashmi, SIEVE is simpler than LRU: an efficient Turn-Key eviction algorithm for web caches, in: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), USENIX Association, 2024, pp. 1229–1246.
URL <https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo>
- [4] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, K. V. Rashmi, Fifo queues are all you need for cache eviction, in: ACM SIGOPS 29th

Symposium on Operating Systems Principles, SOSP '23, 2023.
doi:10.1145/3600006.3613147.

URL https://www.pdl.cmu.edu/PDL-FTP/Storage/FIFOqueues-SOSP23_abs.shtml

- [5] H. Xia, W. Nixon, B. D. Marthen, P. Bhandari, J. Yang, Learning-augmented heuristics: Simple, yet smart, robust and interpretable cache eviction, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026.
URL <https://www.usenix.org/conference/osdi26/presentation/xia>
- [6] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, G. Narasimhan, Driving cache replacement with ml-based lecar, in: Proceedings of the USENIX Workshop on Hot Topics in Storage Systems (HotStorage), 2018.
URL <https://www.usenix.org/conference/hotstorage18/presentation/vietri>
- [7] Z. Song, D. S. Berger, K. Li, W. Lloyd, Learning relaxed belady for content distribution network caching, in: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), USENIX Association, Santa Clara, CA, 2020, pp. 529–544.
URL <https://www.usenix.org/conference/nsdi20/presentation/song>
- [8] E. Liu, M. Hashemi, K. Swersky, P. Ranganathan, J. Ahn, An imitation learning approach for cache replacement, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 6237–6247.
URL <https://proceedings.mlr.press/v119/liu20f.html>
- [9] J. Yang, Z. Mao, Y. Yue, K. V. Rashmi, GL-Cache: Group-level learning for efficient and high-performance caching, in: 21st USENIX Conference on File and Storage Technologies (FAST 23), USENIX Association, Santa Clara, CA, 2023, pp. 115–134.
URL <https://www.usenix.org/conference/fast23/presentation/yang-juncheng>

- [10] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, G. Narasimhan, Learning cache replacement with cacheus, in: 19th USENIX Conference on File and Storage Technologies (FAST 21), USENIX Association, 2021, pp. 341–354.
URL <https://www.usenix.org/conference/fast21/presentation/rodriguez>
- [11] W. Zhou, Z. Niu, Y. Xiong, J. Fang, Q. Wang, 3L-Cache: Low overhead and precise learning-based eviction policy for caches, in: 23rd USENIX Conference on File and Storage Technologies (FAST 25), USENIX Association, Santa Clara, CA, 2025, pp. 237–254.
URL <https://www.usenix.org/conference/fast25/presentation/zhou-wenbin>
- [12] Z. Song, K. Chen, N. Sarda, D. Altınbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, R. Gummadi, HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, 2023, pp. 1149–1163.
URL <https://www.usenix.org/conference/nsdi23/presentation/song-zhenyu>
- [13] J. Yang, Y. Yue, K. V. Rashmi, A large-scale analysis of hundreds of in-memory cache clusters at twitter, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), 2020, pp. 191–208.
URL <https://www.usenix.org/conference/osdi20/presentation/yang>
- [14] University of Massachusetts Amherst, Umass trace repository, official trace repository website (2025).
URL <https://traces.cs.umass.edu/>
- [15] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, M. Paleczny, Workload analysis of a large-scale key-value store, in: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2012, pp. 53–64.
- [16] B. Berg, D. S. Berger, S. McAllister, I. Grosof, S. Gunasekar, J. Lu, M. Uhlar, J. Carrig, N. Beckmann, M. Harchol-Balter, G. R. Ganger,

- The CacheLib caching engine: Design and experiences at scale, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020, pp. 753–768.
URL <https://www.usenix.org/conference/osdi20/presentation/berg>
- [17] cacheMon, libCacheSim: A high performance cache simulator and library, <https://github.com/cacheMon/libCacheSim>, accessed 2026-07-03 (2026).
 - [18] cacheMon, A comprehensive open-source cache trace dataset, https://github.com/cacheMon/cache_dataset, accessed 2026-07-03 (2026).
 - [19] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with ycsb, in: Proceedings of the ACM Symposium on Cloud Computing, 2010, pp. 143–154.
 - [20] D. E. Difallah, A. Pavlo, C. Curino, P. Cudré-Mauroux, Oltp-bench: An extensible testbed for benchmarking relational databases, Proceedings of the VLDB Endowment 7 (4) (2013) 277–288.
 - [21] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, Query optimization through the looking glass, and what we found running the join order benchmark, The VLDB Journal 27 (5) (2018) 643–668. doi:10.1007/s00778-017-0480-7.
 - [22] H. Mao, P. Negi, A. Narayan, H. Wang, J. Yang, H. Wang, R. Marcus, M. Khani, H. He, V. Nathan, F. Cangialosi, S. B. Venkatakrisnan, S. Weng, S. Han, T. Kraska, M. Alizadeh, Park: An open platform for learning-augmented computer systems, in: Advances in Neural Information Processing Systems 32, 2019.
URL https://papers.nips.cc/paper_files/paper/2019/hash/5a83f15f2b94ea59206c4312bb9c3d8f-Abstract.html
 - [23] N. Beckmann, H. Chen, A. Cidon, LHD: Improving cache hit rate by maximizing hit density, in: 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18), USENIX Association, Renton, WA, 2018, pp. 389–403.
URL <https://www.usenix.org/conference/nsdi18/presentation/beckmann>

- [24] D. Yang, D. S. Berger, K. Li, W. Lloyd, A learned cache eviction framework with minimal overhead (2023). `arXiv:2301.11886`, doi: 10.48550/arXiv.2301.11886.
URL <https://arxiv.org/abs/2301.11886>
- [25] R. Shahout, R. Friedman, Sketching the path to efficiency: Lightweight learned cache replacement, in: 27th International Conference on Principles of Distributed Systems (OPODIS 2023), Vol. 286 of Leibniz International Proceedings in Informatics (LIPIcs), 2024, pp. 34:1–34:21. doi:10.4230/LIPIcs.OPODIS.2023.34.
- [26] H. Torabi, H. Khazaei, M. Litoiu, A learning-based caching mechanism for edge content delivery, in: Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering, 2024, pp. 236–246. doi:10.1145/3629526.3645037.
- [27] A. Jain, C. Lin, Back to the future: Leveraging belady’s algorithm for improved cache replacement, in: Proceedings of the 43rd International Symposium on Computer Architecture, 2016, pp. 78–89. doi:10.1145/3007787.3001146.
- [28] I. Shah, A. Jain, C. Lin, Effective mimicry of belady’s min policy, in: 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2022, pp. 558–572. doi:10.1109/HPCA53966.2022.00048.
- [29] D. L.-K. Wong, H. Wu, C. Molder, S. Gunasekar, J. Lu, S. Khandkar, A. Sharma, D. S. Berger, N. Beckmann, G. R. Ganger, Baleen: ML admission & prefetching for flash caches, in: 22nd USENIX Conference on File and Storage Technologies (FAST 24), USENIX Association, 2024, pp. 347–371.
URL <https://www.usenix.org/conference/fast24/presentation/wong>
- [30] S. Jiang, F. Chen, X. Zhang, Clock-pro: An effective improvement of the clock replacement, in: Proceedings of the Annual USENIX Technical Conference, 2005, pp. 323–336.
URL <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/clock-pro-effective-improvement-clock-replacement>

- [31] P. Cao, S. Irani, Cost-aware www proxy caching algorithms, in: Proceedings of the USENIX Symposium on Internet Technologies and Systems, 1997, pp. 193–206.
URL <https://www.usenix.org/conference/usits-97/cost-aware-www-proxy-caching-algorithms>
- [32] G. Einziger, R. Friedman, B. Manes, Tinylfu: A highly efficient cache admission policy, *ACM Transactions on Storage* 13 (4) (2017) 35:1–35:31. doi:10.1145/3149371.
- [33] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, *Journal of the ACM* 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.
- [34] A. Wei, Better and simpler learning-augmented online caching, in: Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020), Vol. 176 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 60:1–60:17. doi:10.4230/LIPIcs.APPROX/RANDOM.2020.60.
URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.APPROX/RANDOM.2020.60>
- [35] J. Chłedowski, A. Polak, B. Szabucki, K. T. Żoła, Robust learning-augmented caching: An experimental study, in: Proceedings of the 38th International Conference on Machine Learning, Vol. 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 1920–1930.
URL <https://proceedings.mlr.press/v139/chledowski21a.html>
- [36] S. Im, R. Kumar, A. Petety, M. Purohit, Parsimonious learning-augmented caching, in: Proceedings of the 39th International Conference on Machine Learning, Vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 9588–9601.
URL <https://proceedings.mlr.press/v162/im22a.html>
- [37] P. Chen, H. Zhao, X. Tang, Y. Wang, S. Deng, Towards optimal robustness in learning-augmented paging (2026). arXiv:2606.01342, doi:10.48550/arXiv.2606.01342.
URL <https://arxiv.org/abs/2606.01342>

- [38] H. Wang, H. He, M. Alizadeh, H. Mao, Learning caching policies with subsampling, in: NeurIPS 2019 Workshop on Machine Learning for Systems, 2019.
URL https://mlforsystems.org/assets/papers/neurips2019/learning_wang_2019.pdf
- [39] S. Ross, G. Gordon, D. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, Vol. 15 of Proceedings of Machine Learning Research, PMLR, 2011, pp. 627–635.
URL <https://proceedings.mlr.press/v15/ross11a.html>
- [40] P. Castells, A. Moffat, Offline recommender system evaluation: Challenges and new directions, *AI Magazine* 43 (2) (2022) 225–238. doi:10.1002/aaai.12051.
- [41] Z. Qiu, J. Yang, M. Harchol-Balter, Why increasing the hit ratio can hurt cache throughput, *Queueing Systems* 110 (2) (2026). doi:10.1007/s11134-026-09986-1.
- [42] OptiSys-ZJU, Cache-Coliseum: A benchmark for learning-augmented caching algorithms, <https://github.com/OptiSys-ZJU/Cache-Coliseum>, accessed 2026-09-15 (2025).
- [43] L. A. Belady, A study of replacement algorithms for a virtual-storage computer, *IBM Systems Journal* 5 (2) (1966) 78–101. doi:10.1147/sj.52.0078.